UO ASSISTANT 0.2.0

Scripting & automation guide

A practical manual for JavaScript scripts, macros, hotkeys, mapping and per-shard workflows.

Development edition
16 October 2026

Includes the complete public host API, twelve standalone examples, runtime bounds and a workflow for testing safely without blocking gameplay.

Contents

1. Start here
2. Shard setup and personal data
3. JavaScript essentials
4. Snapshots and freshness
5. Aliases, friends and reusable scripts
6. Action results and refusal handling
7. Spells, skills and the action catalog
8. Targets and manual control
9. Inventory counts and item movement
10. Journal waits and observation
11. Gumps, menus, text entry and prompts
12. Properties, buffs and matching rules
13. Loot, scavenging, vendors and equipment
14. Travel, combat and local render options
15. Potion cooldowns and bandage timing
16. Hotkeys for spells and tasks
17. Recorder and visual macro builder
18. Profiles, layout and backups
19. Passive alerts and bounded crafting
20. Mapping and UO Mapper compatibility
21. Performance and responsive gameplay
22. Debugging and trace navigation
23. Runtime budgets and supported environment
24. Complete host API reference
25. Snapshot and rule reference
26. Porting scripts from other assistants
27. Troubleshooting checklist
28. Testing a script before regular use
29. Reference sources and release scope

1. Start here

This manual describes UO Assistant 0.2.0. The game is a complete GPU client. The assistant opens in a separate window and runs its interface and scripts away from the game rendering loop. Closing the assistant closes its paired game client; closing that client ends the same pair. Keep the assistant open while playing.

To create a script, open the assistant and search for JavaScript editor. Add a JavaScript script, enter a name and run timeout, then edit its source. Use Validate source before saving. The Macros & scripts page retains the recorder and existing task controls. Run once before assigning a hotkey. Stop cancels that run and its owned waits. A stopped or refused action is recorded in the trace.

Start with the read-only example below. Run it after entering the world. Script output appears in the assistant log; it does not send speech to the shard. Every example file in SDK/examples is a separate complete script. Examples use currently loaded facts and never need your account password.

The built-in editor supplies API suggestions and JavaScript parsing. The SDK/assistant.d.ts file supplies optional external editor types. Type declarations are editor help, not a module to import. JavaScript is the executable scripting language; the recorder and visual macro builder provide simpler alternatives.

01-status.js

Read the player and performance snapshot without acting.

```
const s = await assistant.snapshot();
await assistant.log(s.InGame
  ? `${s.Player.Name}: ${s.Player.Hits}/${s.Player.HitsMax} HP`
  : "Enter the world before running gameplay scripts.");
await assistant.log(`Draw FPS ${s.Performance.DrawFps}`);
```

2. Shard setup and personal data

In the launcher, choose the shard client folder or its client.exe. The launcher reads the version and existing Login.cfg endpoint when unambiguous, then lets you edit the host, port, encryption and graphics settings. Use the shard supplied data files, including its maps and artwork. A server connection and matching game data are different requirements.

Use Check setup before launch. Its file checks establish presence, not a guarantee that every asset is correct. Test connection makes one TCP connection to the chosen host and port without logging in or sending a game packet. A successful test proves reachability only. Use the encryption mode required by that shard; do not guess from a different server.

Each shard connection has its own private settings directory. Copy connection setup creates a new profile identity instead of copying authentication, characters or world state. Sign in manually. The public examples and installer contain no account data. Rule backups and diagnostic exports belong to your private profile unless you choose to share them.

The client accepts normal shard assistant restrictions. A script cannot grant a feature the server disables. If an action is refused, inspect the specific reason, the active target, and whether the required item or dialog is loaded.

3. JavaScript essentials

Use const for a value that will not be reassigned and let for a loop counter or changing value. Strings use single or double quotes; template strings use backticks and ${expression}. JavaScript names are case sensitive. Snapshot fields are PascalCase: Player.Hits, Items and InGame. Host API methods use camelCase: snapshot, useObject and waitForTarget.

An assistant method returns a promise. Await every host call in the order it should happen. Top-level await is supported because your script runs inside an asynchronous function. return ends that script. Use if to guard an action and a bounded for loop to repeat it. Avoid overlapping host calls with Promise.all when order, targets or dialogs matter.

Hexadecimal numbers such as 0x0E21 are ordinary numbers. Item graphics, hues and object serials are different identifiers. A graphic describes an item type; a serial identifies one object in this session. An item may disappear, move or be replaced between snapshots. Never assume a serial copied from another shard identifies the intended object.

JavaScript has no reliable exact integer representation above 2^53-1. Some dialog tokens, revisions and feature masks originate as 64-bit values. Treat those snapshot fields as informational. Dialog reply methods retain exact identities inside the host; do not copy a numeric token into a hand-built request.

02-bounded-monitor.js

Take three read-only samples with a pause between them.

```
for (let i = 0; i < 3; i++) {
  const s = await assistant.snapshot();
  if (!s.InGame || !s.Connected) break;
  const percent = s.Player.HitsMax > 0
    ? Math.round(100 * s.Player.Hits / s.Player.HitsMax) : 0;
  await assistant.log(`Sample ${i + 1}: ${percent}% HP`);
  await assistant.sleep(250);
}
```

4. Snapshots and freshness

snapshot() returns a copy of the currently loaded world. Connected and InGame distinguish the transport from an active character. SessionId and WorldEpoch change across connections and world transitions. Sequence orders snapshots and CapturedUtc is capture time. Do not keep a snapshot as if it were a live object: call snapshot() again after a wait or action.

Player holds the character, backpack, coordinates, facet, health, stamina, mana, weight, death and mount facts. Items contains loaded objects and Mobiles contains loaded nearby characters. InventoryTruncated, MobilesTruncated and Features.ItemDetailsTruncated signal incomplete lists. A missing object in a partial snapshot is not evidence that it does not exist.

OwnedInventory, Accessible, Movable and IsContainer qualify inventory objects. Equipment, another character's belongings, the bank, a closed container and nearby ground objects must not be treated as interchangeable. PropertiesComplete indicates whether item properties are complete enough for a property rule. An incomplete property list cannot prove a negative match.

Performance contains DrawFps, FrameP95Ms and ClientCpuPercent. Features.NativeTools supplies sampled performance, map and hotkey status in this version; consult the shipped SDK for its exact fields. These are measurements, not proof that every visible movement frame was newly simulated. Use the frame-time graph as well as FPS.

5. Aliases, friends and reusable scripts

Create a named alias in the assistant by selecting the intended object in the current world. SerialOrAlias arguments accept either a numeric serial or a string naming that saved alias. alias(name) resolves the alias; unresolved or stale selections are refused by actions. Use short descriptive names such as supplyBag, craftingTool or pet.

Built-in self and backpack references are resolved from the current character by the host. Use the alias selector to confirm a custom alias after changing shards or characters. Friends are stored selections; isFriend checks the saved list. It does not infer party membership, ownership, consent or hostility.

Avoid exporting personal serials in scripts you intend to share. Keep scripts generic by using aliases, actual action catalog names and per-shard rules. The examples below require no custom alias unless their description explicitly says so.

03-alias-and-friends.js

Resolve the current character and inspect the friend list result.

```
const me = await assistant.alias("self");
const friend = await assistant.isFriend(me);
await assistant.log(`Self serial ${me}; saved friend: ${friend}`);
```

6. Action results and refusal handling

Gameplay actions return an AssistantResult with RequestId, Accepted, Message and Serial. Accepted means the client accepted a request. It does not mean the shard acknowledged a successful heal, purchase, move, cast or dismount. Observe a later relevant state or journal event if success matters.

A host refusal can throw an error and ends an uncaught script. A returned unsuccessful result should also stop the dependent steps. Wrap only the part of the script that can recover meaningfully in try/catch. Log error.message and return instead of silently retrying a blocked command. Repeated requests can hide the original cause and add latency.

Typical refusal causes are a disconnected world, an unavailable spell or skill, server policy, changed item facts, a manual target cursor, a replaced dialog, stale vendor offers, incomplete inventory and an exhausted run budget. The target status and trace show the last known reason. Stop All is useful when diagnosing an unexpected sequence.

04-open-backpack.js

Request opening the backpack and report the actual request result.

```
try {
  const result = await assistant.openBackpack();
  await assistant.log(`${result.Accepted}: ${result.Message}`);
} catch (error) {
  await assistant.log(`Backpack request stopped: ${error.message}`);
}
```

7. Spells, skills and the action catalog

ActionsAvailable lists the current client's Kind, Id, Name and Group values. Find an action there rather than assuming a number from an Orion, Razor or ClassicAssist script. castSpell(id) and useSkill(id) request actual catalog entries; the native client applies availability, policy and current-world checks.

Casting a targeted spell and replying to its target are separate operations. Await the cast, wait for the new target, then reply using a target method. A manual cursor takes priority and can invalidate the script's lease. Do not steal or continually cancel a cursor the player created.

The action list is also the source for searchable spell and task hotkeys. A key binding to a catalog action avoids a script when you only need one spell or task. Use a script for a conditional sequence, an explicit wait, logging or a small bounded workflow.

05-action-catalog.js

List available actions without casting or using a skill.

```
const s = await assistant.snapshot();
const actions = s.ActionsAvailable.slice(0, 12);
for (const action of actions) {
  await assistant.log(`${action.Group}: ${action.Name} (${action.Kind} ${action.Id})`);
}
```

06-heal-self.js

Find the ordinary Heal spell, request it, and target self only after a fresh cursor.

```
const s = await assistant.snapshot();
if (!s.InGame || s.Player.Dead) return;
const heal = s.ActionsAvailable.find(a =>
  a.Kind === "CastSpell" && a.Name.toLowerCase() === "heal");
if (!heal) { await assistant.log("Heal is not in this shard's action catalog."); return; }
const result = await assistant.castSpell(heal.Id);
if (!result.Accepted) { await assistant.log(result.Message); return; }
if (await assistant.waitForTarget(3000)) {
  await assistant.targetSelf();
} else {
  await assistant.log("No new target arrived; no target reply sent.");
}
```

8. Targets and manual control

waitForTarget(timeoutMilliseconds) waits for and leases a fresh cursor for this run. It returns false when none arrives in time. targetSelf, targetLast, target(alias), targetGround and cancelTarget operate through the host target rules. A replacement cursor, manual intervention, reconnect or world transition invalidates ownership.

targetGround(x,y,z,graphic) uses absolute shard coordinates, not pixels from the game window. X and Y are 0..65535, Z is -128..127 and optional graphic is 0..65535. Facet and visibility still matter. targetLast uses the last target recorded by the native client and remains subject to current target validity.

The assistant shows whether a cursor is active, its kind and the last refusal. If a script stops at targeting, inspect that display. Backpack double-clicks, gump dragging, normal movement and game targeting continue through the native GPU client. Assistant commands should not intercept normal text entry or replace the player's manual cursor.

9. Inventory counts and item movement

count(graphic,hue,container) reports Amount, Complete and Message. Hue null means any hue; omitting the container uses the current backpack. Complete is essential when deciding whether supplies are absent. Loaded inventory may exclude unopened containers. The counter view and rule previews expose the same uncertainty.

useType selects a currently loaded matching owned item. useObject uses an exact serial or alias. moveItem requires an accessible movable loaded non-container stack, enough quantity and an accessible loaded container destination. It pins current item facts and rechecks them before dispatch. Opening a container or setting an alias does not grant access to someone else's inventory.

Do not drag with the mouse while an item workflow is in progress. A held item is a manual action and may cause refusal. Use organizer/restock previews for repeated transfers, specify maximum quantities and examine the planned source and destination before running them.

07-count-bandages.js

Count the conventional bandage graphic and report completeness.

```
const count = await assistant.count(0x0E21, null);
await assistant.log(`Loaded bandages: ${count.Amount}; complete: ${count.Complete}`);
if (!count.Complete) await assistant.log(count.Message);
```

08-move-one-item.js

Move one loaded item into a selected destination alias named supplyBag.

```
const s = await assistant.snapshot();
if (!s.InGame || s.InventoryTruncated || s.HeldItem !== 0) return;
const destination = await assistant.alias("supplyBag");
const item = s.Items.find(i => i.Container === s.Player.Backpack &&
  i.OwnedInventory && i.Accessible && i.Movable && !i.IsContainer && i.Amount > 0);
if (!item) { await assistant.log("No eligible backpack stack is loaded."); return; }
const result = await assistant.moveItem(item.Serial, destination, 1);
await assistant.log(result.Message);
```

10. Journal waits and observation

waitForJournal(text,timeout,afterSequence) matches journal text after a sequence boundary. Capture the greatest current journal sequence before sending the action you want to observe. Without that boundary, a previously loaded matching line can satisfy a wait. Text is a substring and may vary with shard language or scripts.

A journal line is an observation, not automatically proof of a particular item or spell outcome. Different actions can produce the same phrase. Prefer a unique server message and verify relevant state afterward. Do not use a fixed sleep as the only success criterion for a purchase, recipe or heal.

Use small waits in bounded loops. sleep pauses the script without blocking rendering. It is not a real-time scheduling guarantee. Local load, the client snapshot interval and server latency can make a response arrive later than the requested pause.

09-journal-boundary.js

Wait briefly for a new matching message; no game action is sent.

```
const s = await assistant.snapshot();
const after = s.Journal.reduce((last, line) => Math.max(last, line.Sequence), 0);
const observed = await assistant.waitForJournal("finished", 250, after);
await assistant.log(observed ? "A new matching line arrived." : "No new matching line.");
```

11. Gumps, menus, text entry and prompts

waitForGump(serverSerial,timeout) returns a matching currently open or newly arrived gump, or false. A zero filter accepts any current matching gump; use a specific observed server identity when you know it. It retains the exact dialog lease internally. Inspect Text, Choices, Switches, Entries and Truncated before deciding to reply.

replyGump(button,switches,entries) sends one reply through that lease. Button is a numeric button ID, not a screen coordinate. Entries are objects with Id and Text. A visible label is not necessarily the button ID. Use the dialog inspector and trace to identify a supported choice. Truncated or replaced dialogs must be re-observed.

waitForMenu(text,timeout) and replyMenu(choice) work with classic menus. waitForTextEntry and replyTextEntry handle a text-entry dialog. waitForPrompt and replyPrompt handle a current server prompt. A dialog reply is not a general keyboard typing function. Reply text is limited to 239 characters, with additional dialog-specific bounds.

Wait immediately before replying. A user response or a newly arrived identical-looking dialog invalidates the earlier lease. Do not loop the same reply against a saved token. Crafting plans require a newly observed exact gump at each reply step instead of replaying a stale button sequence.

10-inspect-gump.js

Inspect an open gump without replying to it.

```
const dialog = await assistant.waitForGump(0, 250);
if (!dialog) { await assistant.log("No gump observed."); return; }
await assistant.log(`Gump ${dialog.ServerSerial}; truncated: ${dialog.Truncated}`);
for (const text of dialog.Text.slice(0, 4)) await assistant.log(text);
```

12. Properties, buffs and matching rules

properties(serial) returns a currently cached ItemDetail or false. It does not force an object-property network query. Read PropertiesComplete before using text or numeric properties to select an item. PropertyRevision changes when the observed details change.

buffs() returns loaded buff facts: Id, Graphic, Text and RemainingMilliseconds. A buff list may not expose a shard's custom effect. It does not include every possible internal server cooldown. A missing buff cannot prove a potion is ready.

Rules can combine property predicates with AND or OR groups and assign priorities. Preview explanations identify incomplete data and why a candidate matched or failed. Numeric property recognition depends on the observed property text; use the exact form shown by your shard. Do not assume translated names share numeric syntax.

11-buffs-and-properties.js

Read buffs and one loaded item property record.

```
const buffs = await assistant.buffs();
await assistant.log(`Loaded buffs: ${buffs.length}`);
const s = await assistant.snapshot();
const item = s.Items.find(i => i.OwnedInventory && i.Accessible);
if (item) {
  const detail = await assistant.properties(item.Serial);
  await assistant.log(detail
    ? `Properties complete: ${detail.PropertiesComplete}`
    : "No properties loaded.");
}
```

13. Loot, scavenging, vendors and equipment

The loot and scavenge APIs take a rule object, preview current candidates internally, then send bounded item requests. They are active operations, not read-only previews. The return value Requested counts requests; it is not a server-confirmed number of items acquired. Use a selected destination, range, amount, delay and total run limit.

vendorBuy and vendorSell likewise build a plan from an observed vendor list and execute its requests. Although their return type is named AssistantPreview, these methods are active. Set MaximumTotal and price/quantity bounds. ManualSelection, truncated offers and changed vendor identities prevent unsafe replay. KeepAmount protects a quantity when selling.

Vendor rules use Match with Graphic and optional Hue, DesiredAmount, optional MaximumUnitPrice, KeepAmount and Properties. A bare overlapping graphic selector can be ambiguous and is refused. Property-qualified priority rules can select distinct candidates without counting one offer twice. Inspect the UI preview before building a script for the same rule.

Use the equipment page for saved dress sets, layer conflicts and two-handed checks. Use organizer and restock pages for repeated container plans. These saved rule pages are not all exposed as separate JavaScript methods. A script must use the documented API rather than inventing calls such as assistant.restock or assistant.dress.

14. Travel, combat and local render options

walkTo uses the native pathfinder over currently loaded nearby terrain. It does not promise a route across an unloaded map, locked building, blocked mobile or a shard-specific teleporter. Its stopping range is 0..3 and its timeout is 1..30000 ms. stopWalking ends assistant movement; stopping a script also cancels that run's movement.

attack, warMode, primaryAbility, secondaryAbility and toggleFlight request native combat actions. Available abilities, flight eligibility, current target and server policy still apply. These calls do not calculate combat decisions or infer friendly consent. Check the relevant loaded mobile and saved friend selection before an attack routine.

setRenderOptions accepts a complete options value. Omitted fields use defaults, so treat it as a full replacement rather than a patch. Defaults are CustomLight false, LightLevel 0, Weather true, Sound true, SoundVolume 100, Music true, MusicVolume 100, Footsteps true, HideVegetation false, TreeStumps false and DeathScreen true. Preserve a copied full current Rendering value when changing only one field.

mapWaypoint(x,y,name) sets the native map's temporary destination. It is not a persistent marker-library insertion. Use the map tools page to save a permanent categorized marker.

12-map-current-location.js

Open the map and set a temporary waypoint at the current position.

```
const s = await assistant.snapshot();
if (!s.InGame) return;
await assistant.openMap();
await assistant.mapWaypoint(s.Player.X, s.Player.Y, "Current location");
```

15. Potion cooldowns and bandage timing

Cooldown definitions belong to the shard profile. Configure a label, a tracked item graphic/hue or action, a duration and a group. Grouped entries share a ready boundary where configured. Ready bars show remaining local time and optional ready alerts. Custom durations let a shard use different potion, consumable and ability rules.

Observed timing suggestions use eligible observed use and journal facts. They are suggestions for explicit review, not automatic edits to your rules. Missing packets, a late capture, a reconnect or a gap in observed journal history can make a timing unusable. A local ready bar is an estimate and the server remains authoritative.

Bandage healing can use a fixed delay or the editable dexterity formula: clamp(base milliseconds - dexterity * milliseconds per dexterity, minimum, maximum). The formula requires a known dexterity value; unknown dexterity is refused rather than guessed. Choose self or a selected pet serial as the healing target. Recheck the selected pet when changing characters.

The UI exposes these timings directly. There is no JavaScript startCooldown, bandageHeal or editTimer method in this release. To script a bandage use, useType the shard's actual bandage graphic, wait for its fresh target and reply to your selected target. Set a conservative timeout and observe the shard response before repeating.

16. Hotkeys for spells and tasks

The hotkey page searches the client's action catalog and saved macros/scripts. Capture a key combination, inspect the action name, and save it to the active shard rules. The assistant normalizes modifiers and reports duplicate, reserved and native macro collisions. Resolve a collision explicitly instead of relying on whichever handler runs first.

A master enable switch disables assistant hotkeys without deleting them. Pass-through lets an accepted binding also reach the native input path where configured. Text and chat entry retain priority: a binding should not cast a spell while you are typing. Test a new combination with the chat field open and closed before using it in play.

Use a direct spell binding for one cast, a macro for a repeatable recorded sequence, and a script when you need conditional logic or observation. Stop controls and manual cursor priority remain in effect. A hotkey does not bypass the action queue, target leases, rule validation or shard restrictions.

The design uses the useful command trees, key capture and master/pass-through controls found in the supplied assistants. Existing Orion, Razor, Injection and ClassicAssist scripts are different languages and APIs; a similar feature name does not make their source directly executable here.

17. Recorder and visual macro builder

Record a short manual sequence, stop recording, name it and inspect each captured step. Recording does not mean every UI interaction can be reproduced safely. Save only intended actions, and confirm item aliases and dialog choices. Keep the original recorded macro separate while experimenting with a visual program.

The visual builder supports action blocks, conditions and bounded repeat blocks. Reorder blocks using the supplied controls and validate the complete program before running. Conditions include health, mana, stamina and weight percentages, loaded item counts, target activity and journal text. Choose explicit comparison values.

Program validation limits nesting, expansion and repetitions. The runtime rechecks conditions against current snapshots. Loops have finite repeat counts; they are not unattended endless automation. A manual target or a replaced dialog can stop a block even if an earlier condition was true.

A normal recorded macro allows 1..100 repetitions, at most 256 recorded steps and at most 2048 expanded actions. Visual programs apply their own validated expansion bounds too. Break a long workflow into named tasks rather than attempting to evade a limit with nested repetitions.

18. Profiles, layout and backups

Play-style profiles can capture selected rules, hotkeys and assistant layout. Use them for a healer, gatherer or crafter setup. Switching a profile changes saved definitions; it does not start playback or sign in. A profile is not a copy of the server account or a world save.

The assistant window supports navigation search and favourites, compact dashboard, size and text scaling, contrast and colour-labelled states. Save a useful layout at the display size you actually use. Placement recovery brings a saved off-screen window back into a visible working area. Docking remains a separate window beside the game.

Automatic rule backups rotate a configured count of saved definitions. The backup list and restore preview let you inspect a previous version before replacing current rules. Restore affects assistant rules, not shard files or game world saves. Save conflicts are refused when another instance changed the same definitions; reload and review rather than overwriting blindly.

Export rules for transfer only after reviewing their aliases, paths and marker labels. Personal rule exports may contain information you entered. Public installer examples are generic. Update recovery backs up assistant definition files separately before starting an explicitly chosen installer.

19. Passive alerts and bounded crafting

Passive alerts monitor low health, low mana, high weight, low supplies, durability or disconnection using available facts. Choose thresholds, quiet intervals and optional sounds. An alert reports a condition; it does not send an item use, spell or chat message. Supply and durability alerts require complete relevant observations.

A crafting plan names a selected owned tool, ingredient minima, a maximum attempt count, delays, exact gump IDs and reply buttons, and success/failure journal messages. The plan checks ingredients before each leased reply and requires a newly observed exact gump for each reply step. It does not click arbitrary screen coordinates.

Success/failure messages must match your shard's actual system messages. If the observation window expires, the plan reports an unknown result and does not retry blindly. Stop-on-failure stops further attempts when the observed failure phrase appears. Ingredient presence is not proof that a particular recipe is available.

Crafting plans run through their coordinator/UI in this release. There is no assistant.craft JavaScript method. For a custom script, build a small explicit useObject/waitForGump/replyGump/waitForJournal sequence and retain the same bounded, fresh-dialog and outcome checks.

20. Mapping and UO Mapper compatibility

The supplied ultima-mapper archive is a mapper for Ultima IV. Its dungeon and WORLD.MAP formats are not Ultima Online client data. This version instead integrates the existing UO GPU world map with a private searchable marker library, categories, visibility filters and current-position markers.

Marker tools support the recognized UOAM .map, Ultima Mapper XML and CSV formats. Imports are bounded, validate coordinates/facets and disable XML DTD processing. Review the import result and category before saving. Export writes selected marker data only; it does not convert or redistribute the shard's terrain artwork.

ClassicUO marker CSV fields are x,y,map,name,icon,color,zoom. Ultima Mapper XML uses Marker elements with X, Y, Name, Facet and Icon attributes. UOAM marker lines use a signed icon prefix with coordinates and a facet. Use the supplied export to produce a valid file instead of guessing quoting rules for commas or special characters.

Route preview uses loaded nearby terrain. A preview is informational and does not move your character. Persistent map markers and temporary mapWaypoint destinations are separate. This implementation does not embed the third-party Ultima Mapper executable, connect its external sharing service or reproduce its proprietary interface.

21. Performance and responsive gameplay

The client GPU renderer owns the whole game window. Scripts, rule editing, map imports, diagnostics and assistant navigation should not become frame-by-frame game work. Native snapshots and timing samples are bounded, and background monitoring is sampled rather than continuously scanning every object.

The performance page displays Draw FPS and recent frame-time samples alongside client process CPU, memory and native ping. Stage samples help distinguish update/render work from total frame pacing. GPU and VRAM measurements are optional OS observations; unsupported counters show unavailable instead of a fabricated zero.

60 FPS corresponds to about 16.7 ms per displayed frame. A high average FPS can coexist with visible stutters; compare the frame-time graph and p95 value while running, dragging gumps and casting. Ping measures network response rather than graphics speed. A loaded disk or asset decode can affect a frame without a high steady CPU percentage.

For a useful test, start with the shard's correct assets and default 60 FPS with VSync. Walk and run through a repeatable area, use the backpack and target a world object. Then enable one additional agent or script at a time. Keep loops bounded, avoid logging every frame and use waits between observations. Diagnostic export is opt-in and intended to omit credentials, chat text and personal paths; inspect any file before sharing it.

22. Debugging and trace navigation

Validate catches JavaScript syntax errors before execution. A parse error identifies a user source line and column; fix it before investigating the game. API completion is derived from the supported host methods. Suggestions do not imply that arbitrary browser, Node.js or .NET APIs are available.

Runtime trace records action boundaries, request results and cancellation/refusal. The editor links to Script tools for that trace; parser errors navigate to the source line. Runtime operation entries do not have JavaScript source maps or arbitrary statement stepping in this release. Match the first failure to the current target, dialog, inventory or connection state. Subsequent steps often never ran, so fixing their parameters will not solve the first failure.

Use log for concise facts and branch decisions. Log the completeness flag along with a count, or the dialog identity along with a reply choice. Avoid passwords, private journal content, account details and long object dumps. A trace exported for another person can reveal your chosen labels or workflow.

If an item action fails, open the container and inspect ownership/access. If a target fails, check whether a manual cursor replaced it. If a vendor action fails, reopen the vendor list and preview again. If a script times out, reduce its work and observe a specific event rather than increasing every timeout.

23. Runtime budgets and supported environment

Each JavaScript run uses its own Jint engine. The language is JavaScript with a deliberately small host API. There is no CLR reflection, file access, process launch, network fetch, module import, require, document/window browser DOM or arbitrary native code. Use the assistant UI for importing files and exporting markers.

A script source is limited to 65536 characters. A configured run lasts 1..300 seconds, with a statement budget of 250000, 16 MiB engine memory and recursion limit 64. A run permits at most 1024 host operations. Host arguments are bounded JSON with at most six arguments and a 4096-byte request limit. Snapshots are capped at 1 MiB.

sleep and wait timeouts are generally 0..30000 ms. walkTo uses 1..30000 ms. Journal search text is limited to 256 characters and log text to 512. Queue delay is 100..30000 ms. These limits bound work and aid cancellation; they are not suitable for a perpetual background bot.

Stop, disconnect, a changed world identity or an expired run invalidates owned operations. Fire-and-forget calls are still observed by the worker during shutdown, but writing every intended call with await makes ordering understandable and avoids an action outliving the line you expected to control it.

24. Complete host API reference

The following is the complete public assistant method set for this release. All methods return promises and should be awaited. Methods that act return AssistantResult unless another return value is stated. Numeric range checks are additional to current-world and server-policy checks.

snapshot()

Return the currently loaded AssistantSnapshot; read-only. Refresh after waits and actions.

alias(name)

Resolve a saved or supported built-in alias to a serial; confirm custom selections per character.

isFriend(serialOrAlias)

Return whether the serial is in the saved friend list; not a hostility or party test.

count(graphic, hue = null, container = backpack)

Return Name, Amount, Complete, Message for loaded matching items. Graphic/hue: 0..65535.

log(text)

Write up to 512 characters to the assistant log; does not send speech.

sleep(milliseconds)

Yield this script for 0..30000 ms, cancellable and subject to the overall run timeout.

waitForTarget(timeout)

Wait 0..30000 ms for a fresh run-owned target; return true or false.

waitForJournal(text, timeout, afterSequence = null)

Wait for matching journal text, optionally after an explicit sequence boundary. Text: up to 256 characters.

openBackpack()

Request the current player backpack gump.

openMap()

Open the native world map.

dismount()

Request dismount through the native client; observe Mounted afterward.

useObject(serialOrAlias)

Request use of a currently valid object.

useType(graphic, hue = null)

Use a matching loaded owned item type, subject to current availability.

castSpell(id)

Request an available spell catalog ID; host numeric bound 1..10000, native availability still required.

useSkill(id)

Request an available skill catalog ID; host numeric bound 0..1000.

targetSelf()

Reply to the run-owned target with the current player.

targetLast()

Reply using the current valid native last target.

target(serialOrAlias)

Reply with the selected current entity serial.

cancelTarget()

Cancel a permitted current target without taking over a manual cursor.

targetGround(x, y, z, graphic = 0)

Reply with absolute ground coordinates and optional graphic, subject to facet/target rules.

moveItem(serialOrAlias, destination, amount)

Request 1..65535 units from a loaded eligible non-container stack to a loaded accessible container.

properties(serialOrAlias)

Return cached AssistantItemDetail or false; never force a server property query.

buffs()

Return the currently loaded AssistantBuff array.

waitForGump(serverSerial = 0, timeout = 5000)

Return an open/new matching dialog or false; retain exact lease for replyGump.

waitForMenu(text = "", timeout = 5000)

Return a matching classic menu or false; retain lease for replyMenu.

waitForTextEntry(timeout = 5000)

Return a text-entry dialog or false; retain lease for replyTextEntry.

waitForPrompt(timeout = 5000)

Return a prompt or false; retain exact prompt identity internally.

replyGump(button, switches = [], entries = [])

Reply once to the retained exact gump. Entries: {Id, Text}; button IDs are not coordinates.

replyMenu(choice)

Reply to the retained classic menu; numeric choice 0..65535 with native dialog validation.

replyTextEntry(text, cancel = false)

Reply to retained text entry; text up to 239 characters and any smaller dialog limit.

replyPrompt(text, cancel = false)

Reply to retained server prompt; text up to 239 characters.

vendorBuy(rules)

Plan AND SEND bounded purchase requests from the observed vendor list; returns a plan-shaped summary.

vendorSell(rules)

Plan AND SEND bounded sale requests; honor KeepAmount and completeness/current-offer checks.

loot(rules)

Plan and request bounded corpse loot transfers; returns {Requested}, not acknowledgement.

scavenge(rules)

Plan and request eligible ground transfers using current rules/range; returns {Requested}.

walkTo(x, y, z, stoppingRange = 1, run = false, timeout = 10000)

Run native nearby loaded-terrain pathfinding; stop range 0..3, timeout 1..30000. Returns true or throws.

stopWalking()

Stop assistant movement through the native client.

attack(serialOrAlias)

Request attack on a currently valid target.

warMode(enabled)

Set requested native war/peace mode.

primaryAbility()

Request the currently available primary weapon ability.

secondaryAbility()

Request the currently available secondary weapon ability.

toggleFlight()

Request eligible native flight toggle.

setRenderOptions(options)

Replace the full local rendering/audio options value; omitted fields take defaults.

mapWaypoint(x, y, name)

Set a temporary map destination, name up to 80 characters; not persistent marker storage.

25. Snapshot and rule reference

AssistantSnapshot: Schema, SessionId, WorldEpoch, Sequence, CapturedUtc, InGame, Connected, Status, Player, Performance, Target, HeldItem, Items, Mobiles, Journal, ActionsAvailable, Actions, Capabilities, InventoryTruncated, MobilesTruncated and Features. See SDK/assistant.d.ts for exact optional and new native fields.

Player: Serial, Name, Backpack, X, Y, Z, Map, Hits/HitsMax, Mana/ManaMax, Stamina/StaminaMax, Weight/WeightMax, Dead, Mounted and the observed dexterity field. Item: Serial, Graphic, Hue, Amount, Container, Layer, Name, X/Y/Z, OwnedInventory, Accessible, Movable, IsContainer.

Mobile: Serial, Name, Body, Hue, X/Y/Z, Distance, Notoriety and any observed health facts. Journal: Sequence, Time, Text, Type, Name. ActionsAvailable: Kind, Id, Name, Group. Target: Active, Revision, Id, Type, LastSerial. Action history includes origin and captured item-type facts when available.

Features: ItemDetails, ItemDetailsTruncated, Buffs, Dialogs, Vendors, Prompt, Movement, Combat, Rendering, PartyMembers, LockedFeatures, CharacterFeatures, AssistantPolicy and NativeTools. Some features are observational and cannot be assigned from a script.

ItemMatch: {Graphic, Hue?}. PropertyRule: {Name?, Minimum?, Maximum?, Contains?}. Loot rules: Name?, Source?, Destination, MaximumRange?, DelayMilliseconds?, MaximumRunSeconds?, OpenCorpses?, Rules. Each loot entry can use Match, NameContains, Properties and MaximumAmount.

Vendor rules: Name?, Vendor?, MaximumTotal?, Rules. Each vendor entry uses Match and DesiredAmount, with optional MaximumUnitPrice, KeepAmount and Properties. Rule priorities/property groups added to the saved UI schema should be created through the rule editor and preview, using SDK types for script support.

AssistantDialog: Token, Revision, LocalSerial, ServerSerial, Kind, Text, Choices, Switches, Entries, Truncated. Vendor: Token, Revision, Serial, Buy, Truncated, ManualSelection, Offers. These identity fields describe observed state; replies keep exact host leases rather than accepting arbitrary tokens.

26. Porting scripts from other assistants

Razor and Injection command scripts, Razor Enhanced and ClassicAssist Python APIs, and Orion JavaScript APIs are separate environments. Do not paste them into this editor expecting identical behavior. First identify the intent: use an item, cast, wait for a cursor, select a target, move an item or reply to a specific gump.

Map a source command to a documented method in chapter 24. Replace global serial constants with aliases and source spell numbers with ActionsAvailable lookups. Replace polling loops with a bounded wait and explicit timeout. Keep the manual player's target priority and retain fresh dialog leases.

The existing legacy import supports only its explicitly recognized subset. Inspect the import report for every skipped or unsupported instruction. An imported task is not verified until you inspect and test it. There is no general Python interpreter or guaranteed one-to-one Orion compatibility in this release.

Features from the reference assistants were adopted where they fit a responsive GPU client: searchable action hotkeys, item and property rules, agents, aliases, counters, organized maps, script tooling and shard profiles. Implementations and names differ; consult this release's API instead of documentation for another program.

27. Troubleshooting checklist

Syntax error: validate, go to the indicated source line, check braces/quotes and verify await is inside the supported script body. Unknown method: compare the name with chapter 24 and the SDK; do not assume another assistant's API exists.

No matching item: enter the world, open its container, inspect graphic/hue and ownership, refresh the snapshot, and check completeness. Target refusal: confirm a fresh cursor arrived for this run and no manual action replaced it. Gump refusal: inspect the current gump ID and await a fresh lease before replying.

Count seems wrong: the loaded inventory is incomplete, nested containers may be closed, or the container/hue filter differs. Timer seems wrong: local bars are estimates; inspect configured shard duration, group and observed event, then adjust explicitly. Path blocked: choose a nearer loaded destination and inspect the route preview.

Hotkey does nothing: check master enable, active profile, text/chat focus, reserved/native conflicts and action availability. Stutter: compare frame-time samples, temporarily stop new scripts/agents, reduce logging/import work, and repeat a movement path. Reconnect: refresh aliases and observations; a previous world's leases cannot be reused.

Update check fails: the feed is optional and must be a direct HTTPS signed release catalog. No hosted feed is configured by default. You can open the matching local signed catalog and verify its installer manually. The development installer is unsigned for Windows publisher identity; release-catalog signatures are a separate integrity mechanism.

28. Testing a script before regular use

1. Save current rules and inspect a backup. Use a test character or local shard for a new action sequence. Confirm shard rules permit the task. 2. Validate syntax. Run a read-only snapshot/count/dialog-inspection version first. 3. Add one action and inspect its result and the actual game response.

4. Add a bounded wait with a fresh sequence or target/dialog lease. 5. Test absence, incomplete inventory, manual cursor intervention and disconnect. A useful script stops clearly in these cases. 6. Add a finite repeat count only after one iteration succeeds. 7. Assign a hotkey and test chat focus and conflicts.

8. Verify movement, gump dragging and targeting stay responsive while the script runs. Compare frame-time samples before and during it. 9. Export only generic definitions and remove personal aliases or labels before sharing. 10. Recheck after a shard changes its client version, dialogs, messages, timings or assistant policy.

The included example files are exercised offline against the actual script engine with owned synthetic world facts. That checks syntax, host dispatch and bounded behavior; it does not certify a particular live shard's spell names, recipes, delays or permissions. The final gameplay acceptance test is your manual shard session.

29. Reference sources and release scope

Local reference archives: OrionUO-master, RazorEnhanced-release-1.0, ClassicAssist-develop, Injection_0.3.30.2-master, Razor-master and ClassicUO-main. The separate ultima-mapper-master README describes Ultima IV input files. Archive documents are research material, not instructions to modify user accounts or assets.

Primary mapping references: ClassicUO World Map documentation, https://github.com/ClassicUO/ClassicUO/wiki/World-Map ; Ultima Mapper project site, https://www.ultimamapper.com/ . The implemented marker readers follow the formats recognized by the UO client and validate imported data locally.

Primary feature references: Razor Enhanced hotkeys documentation, https://www.razorenhanced.net/dokuwiki/doku.php?id=enhanced_hotkey_tab ; bandage agent documentation, https://www.razorenhanced.net/dokuwiki/doku.php?id=bandage_heal_agnet . The shipped source and tests define this application's behavior.

This development release includes the executable application, a separate assistant, this guide, generic examples, SDK types and applicable notices/source-rebuild material. It does not contain Ultima Online game assets, shard accounts or a hosted update service. It does not install or modify a live shard. Choose your installed shard data folder when setting up the launcher.